VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgaum, Karnataka – 590 018

A Mini-Project report on

"Design of Radix-8 Booth Multiplier using Verilog and Implementation on De10 Cyclone V FPGA"

Submitted in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Engineering

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Manjegouda O Y	**USN:1KI23EC038**
Nithin Kumar M	**USN:1KI23EC053**
Parashuram Ramesh Telkar	**USN:1KI23EC056**
Pavan V K	**USN:1KI23EC057**

Under the guidance of

Prof. PradeepKumar S K B.E.,M.Tech.,(PhD)
Assistant Professor
Dept. of ECE
K.I.T,Tiptur.

Head of the Department

Dr. S V Rajashekararadhya M.E.,Ph.D
Professor& H.O.D
Dept. of ECE
K.I.T,Tiptur.

Principal

Dr. H. C. Sateesh Kumar BE., M.Tech., Ph.D
K.I.T,Tiptur.

Department of Electronics and Communication Engineering
Kalpataru Institute of Technology
Tiptur– 572202

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgaum, Karnataka – 590 018

KALPATARU INSTITUTE OF TECHNOLOGY

Department of Electronics and Communication Engineering

TIPTUR-572202

CERTIFICATE

This is to certify that the miniproject work entitled **"Design of Radix-8 Booth Multiplier using Verilog and Implementation on De10 Cyclone V FPGA"**is successfully carried out by **Manjegouda O Y (1KI23EC038), Nithin Kumar M(1KI23EC053), Parashuram Ramesh Telkar (1KI23EC056), Pavan V K (1KI23EC057),**thebonafide students of Electronics & Communication Engineering Department, Kalpataru Institute of Technology, Tiptur in partial fulfillment for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering of Visvesvaraya Technological University, Belgaum,during the year 2025-26. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Under the guidance of

Prof. PradeepKumar S K B.E.,M.Tech.,(PhD)
Assistant Professor
Dept. of ECE
K.I.T,Tiptur.

Head of the Department

Dr. S V Rajashekararadhya M.E.,Ph.D
Professor & H.O.D
Dept. of ECE
K.I.T,Tiptur.

Principal

Dr. H. C. Sateesh Kumar BE., M.Tech., Ph.D
K.I.T,Tiptur.

External Viva

Name of the Examiners	Signature with Date
1._____	_____
2._____	_____

Abstract

Multiplication is a fundamental arithmetic operation used in several digital systems including DSP processors, computer ALUs, cryptographic engines and hardware accelerators. This project focuses on the design and FPGA implementation of a 32-bit Radix-8 Booth Multiplier using Verilog HDL. Radix-8 Booth encoding reduces the number of partial products significantly by processing three bits of the multiplier at a time, thereby improving computational efficiency compared to conventional binary multiplication methods.

The proposed hardware architecture is functionally verified using ModelSim simulator and further implemented on the Cyclone-V based DE10-Standard FPGA development board using Quartus Prime Lite Edition. Functional correctness is validated through SignalTap on-chip logic analyzer. The results show that the designed multiplier achieves correct signed arithmetic computation for all tested operand combinations with only 4% logic utilization on FPGA and without using any DSP blocks. Hence, this Radix-8 Booth multiplier architecture is proven to be efficient, fast and suitable for high-performance arithmetic applications in modern digital VLSI hardware systems.

ACKNOWLEDGEMENT

This mini project consumed huge amount of work, research and dedication. After an intensive period of six months, today is the day: writing this note of thanks is the finishing touch on our dissertation. We would like to reflect on the people who have supported and helped us so much throughout this period.

Firstly, we would like to express our deepest gratitude to our project guide **Prof. PradeepKumar S K**, Asst. Professor, Department of E&CE whose guidance, expertise, and encouragement were invaluable throughout this work specially for his support during simulation and hardware testing. We attribute the level of our Bachelor degree to his encouragement, effort and without him this thesis, too, would not have been completed or written. One simply could not wish for a better or friendlier academic advisor.

We would like to thank our Head of the Department, **Dr. S V Rajashekararadhya,** Department of E&CE, K.I.T, Tiptur, for his hearty guidance and constant support. We also thank the Department of Electronics & Communication Engineering for providing access to the required FPGA development board, software tools (Intel Quartus Prime, ModelSim, SignalTap), and laboratory facilities

We are indebted to our principal, **Dr. H. C. Sateesh Kumar**, Kalpataru Institute of Technology, Tiptur, for his support and encouragement for our mini project. With great respect and obedience, we thank our parents and family members who were the backbone behind our deeds.

Project Associates

ManjeGouda O Y
Nithin Kumar M
Parashuram Ramesh Telkar
Pavan V K

Table of Contents

List of Figures

List of Tables

CHAPTER -1

INTRODUCTION

1.1 Background

Arithmetic circuits play a vital role in Digital Signal Processing (DSP), control systems, image processing and embedded computing applications. Among them, multiplication is one of the most frequently used arithmetic operations in hardware systems. High performance multipliers are required to achieve better system throughput and improved computation speed. With increase in data size and algorithmic complexity, conventional multiplication techniques become slower and resource heavy in FPGA and VLSI platforms.

1.2 Need for Fast Multipliers

As modern processors and digital accelerators demand high speed mathematical operations, multiplier design becomes a primary optimization factor. Large word-size multipliers (32-bit or higher) directly impact system delay, power, and chip-resource utilization. Hence, optimized multiplication algorithms must be applied to reduce latency and improve computation efficiency.

1.2.1 Booth Algorithm and Radix-8 Optimization

Booth multiplication is one of the most widely used signed multiplication algorithms. It reduces the number of partial products by encoding multiple multiplier bits at a time. Radix-8 Booth algorithm processes 3 bits of the multiplier at once, therefore it reduces the number of partial products by almost **3x** compared to normal multiplication. This reduction significantly improves performance and area efficiency when implemented on FPGA.

1.2.2 Verilog HDL and FPGA Implementation

Verilog HDL is a Hardware Description Language widely used for designing digital systems at register-transfer level (RTL). FPGA (Field Programmable Gate Array) platforms provide reconfigurable hardware, which allows rapid prototyping and real-time implementation of digital circuits before ASIC fabrication. In this work, the design is

implemented on Cyclone-V based DE10-Standard FPGA Boardusing Intel Quartus Prime Lite software

1.3 Problem Statement

To design, implement and evaluate a 32-bit Radix-8 Booth Signed Multiplier using Verilog HDL and to verify the correctness through simulation as well as real FPGA hardware execution.

1.3.1 Objective of the Project

- To study the theory of Radix-8 Booth Multiplication.
- To develop Verilog RTL code for a 32-bit Booth Multiplier.
- To simulate the design using ModelSim and verify functional correctness.
- To perform FPGA synthesis and implementation on Cyclone-V (DE10-Standard).
- To analyze device utilization, timing and performance parameters.

1.4 System Level Block Diagram

The overall functional structure of the proposed Radix-8 Booth multiplier system is shown in Figure 1.1. The two signed 32-bit input operands (A and B) are applied to the Radix-8 Booth multiplier core. The multiplier unit processes the inputs using Booth recoding and sequential partial product accumulation technique. After completion of computation, a 64-bit signed product result is generated and displayed. The design is implemented in Verilog HDL and deployed on Cyclone-V DE10-Standard FPGA.

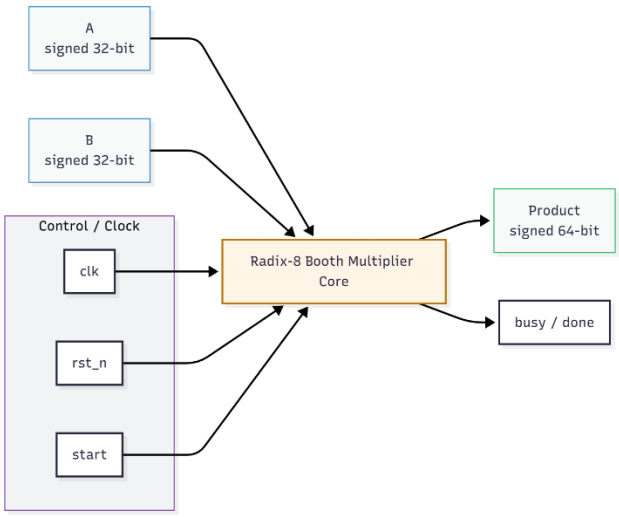

Figure 1.1. The two signed 32-bit input operands (A and B) are applied to the Radix-8 Booth multiplier core.

1.5 Organization of the Report

The remaining chapters in this project report are organized as follows:

- **Chapter 1** presents the introduction, motivation, problem definition and overall scope of the Radix-8 Booth Multiplier design project.
- **Chapter 2** provides the detailed literature survey related to multiplier architectures, Booth algorithm theory, existing multiplier implementations and research contributions in this domain.
- **Chapter 3** explains the theoretical background of Radix-8 Booth encoding, mathematical formulas, Booth recoding process and internal working principle of the multiplication algorithm.
- **Chapter 4** describes the complete methodology, system architecture, RTL design structure, FSM modelling and internal block level operation of the proposed multiplier design.
- **Chapter 5** presents the FPGA synthesis and implementation results using Quartus Prime targeting Cyclone-V device, resource utilization report and post-fitting technology mapping.
- **Chapter 6** discusses the functional simulation results from ModelSim and SignalTap hardware verification along with output tables, waveform analysis and result validation proof.
- **Chapter 7** provides the performance discussion and comparison between Radix-4 and Radix-8 Booth architectures based on computational efficiency and FPGA resource analysis.

CHAPTER – 2

LITERATURE REVIEW

Multiplication is one of the most fundamental arithmetic operations used in almost all digital signal processors, computer arithmetic units, cryptographic units, filtering systems and VLSI based computation blocks. The performance of most modern high-speed systems depends mainly on how efficiently multiplication is performed in hardware. Over the years, several hardware multiplier architectures have been evolved in order to improve speed, minimize power consumption and reduce silicon area.

In recent years, several researchers have focused on multiplier optimization because arithmetic units determine the overall power and timing performance of modern computing systems. Many published works highlight that enhancing multiplication efficiency significantly improves processor throughput, especially in applications like FIR filtering, matrix computation, convolution engines, hardware neural networks and image processing pipelines. It has been reported that Booth encoding based multipliers provide better balance between speed and hardware utilization compared to full parallel tree multipliers. However, radix-2 and radix-4 Booth implementations still generate considerable intermediate values and require more accumulation cycles. This directly affects clock cycles, switching activity and dynamic power. Therefore, moving to higher radix encoding such as Radix-8 is considered more suitable for FPGA based arithmetic cores where reduction in number of partial products results in improved real-time performance without excessively increasing area complexity.

2.1 Conventional Multiplication Techniques

Traditional shift-and-add multiplication requires generation of multiple partial products and summing them sequentially. Although easy to design, this method is slow for large bit-width operands because it requires one addition for each bit of the multiplier. For 32-bit operands, this may require up to 32 clock cycles of accumulation, resulting in increased computation time.

2.2 Wallace Tree Based Multipliers

Wallace tree multiplier architecture reduces delay by performing parallel partial products generation and using compressor trees to sum the partial products. While this method

provides improved speed, its wiring complexity and hardware resource usage increases significantly for higher data width designs, making it less area efficient for FPGA based implementations.

2.3 Booth Multiplication

Booth algorithm is a widely used method to reduce the number of partial products by encoding the multiplier bits in groups. Traditional Booth algorithm processes 2 bits at a time (Radix-2 Booth Encoding). Later, improved versions like Radix-4 and Radix-8 Booth encoding were introduced to handle multiple bits per iteration.

- **Radix-2 Booth** → operates on 2 bits per cycle
- **Radix-4 Booth** → operates on 2 bits effective but less partial products
- **Radix-8 Booth** → operates on 3 bits at a time, reducing partial products to almost 1/3

2.4 Radix-8 Booth Multiplier Advantage

Radix-8 Booth encoding supports multiplication by multiples of 0, ±1, ±2, ±3, ±4 which reduces number of partial products significantly. This helps in achieving high speed and better resource optimization when implemented on FPGA device. Therefore, Radix-8 Booth multiplier is highly suitable for applications where large-size signed multiplication is required frequently.

2.5 Verilog HDL Synthesis Improvements

FPGA tool chains like Intel Quartus Prime optimize combinational logic, pipelining and resource mapping automatically. Radix-8 Booth algorithm when written in Verilog HDL at RTL level, results in efficient synthesis mapping on Logic Elements (LEs) of Cyclone-V FPGA. The high flexibility of Verilog allows modular development and easy verification using testbench simulation tools like ModelSim

2.6 Research Gap / Identified Need

Most existing low-level Verilog multiplier implementations follow Radix-2 or Radix-4 Booth method which still generate more partial products and thus result in increased latency and resource overhead. Radix-8 implementation solves this by reducing cycles and improving performance. Hence this project focuses on implementing Radix-8 Booth

multiplication using 32-bit signed operation and deploy it on real FPGA hardware to validate its efficiency and correctness.

From the above study, it is clear that Radix-8 Booth multiplier is an optimized approach for high-performance signed multiplication especially in FPGA based digital design applications. This work implements Radix-8 Booth logic using Verilog HDL and evaluates it through synthesis, simulation and FPGA execution

CHAPTER – 3

METHODOLOGY AND RADIX-8 BOOTH ALGORITHM THEORY

3.1 Overview of Multiplication in Digital Systems

Multiplication is one of the most frequently used arithmetic operations in modern processors, digital signal processing cores and embedded computational units. In hardware implementation, multiplication between two N-bit numbers produces a 2N-bit output result. For a 32-bit system, the product becomes 64-bit wide. Conventional shift-and-add based multipliers consume a large number of partial product cycles and become slow for high-performance digital applications. Therefore, optimized multiplication algorithms such as Booth Radix based encoders are preferred in VLSI/FPGA design.

3.2 Signed Multiplication and Two's Complement Handling

Most modern systems operate on two's complement signed binary numbers. In two's complement representation:

- Positive values remain unchanged
- Negative values are stored in form of inverted bits + '1'

Radix-8 Booth multiplication supports signed binary multiplication naturally, since it interprets the multiplier bits along with an additional sign extension bit and processes multiple bits at a time. This reduces partial products and supports both positive and negative operands in a unified manner, making it suitable for real hardware implementation like FPGA

3.3 Booth Recoding Theory

Booth recoding is a technique used to minimize the number of partial products generated during multiplication by examining multiple bits of the multiplier at a time. Instead of adding the multiplicand for every single '1' in the multiplier bit sequence, the Booth algorithm analyses patterns of adjacent bits and decides whether to add, subtract or skip the multiplicand based on the bit transitions. In Radix based Booth systems, this bit grouping is extended to handle larger blocks of bits per cycle. For Radix-8 Booth encoding, three bits of the multiplier are taken together along with an additional previous

bit to form a 4-bit group. This 4-bit group is decoded to produce a digit between -4 and +4. Each digit represents how many times the signed multiplicand must be added or subtracted in that cycle. In this way, instead of performing 32 operations for 32-bit multiplication, Radix-8 requires only around 11 iterations. This drastically reduces multiplication latency and makes the algorithm highly efficient for FPGA and high speed arithmetic implementations. Booth recoding also naturally supports signed number multiplication using two's complement representation, which makes it superior and more hardware friendly for real digital systems.

3.3.1 Radix-8 Booth Encoding Process

Radix-8 Booth encoding considers **3 bits of the multiplier** at a time. Each recoded group generates one digit between **-4 to +4**. This digit decides how many times the multiplicand must be added or subtracted and then shifted.

For a 32-bit multiplier, total groups required:

$$Digits = (32 + 2) / 3 \approx 11 \text{ groups}$$

Each group corresponds to one iteration in the accumulator update cycle.

3.3.2 32-bit Signed Booth Radix-8 Mathematical Derivation

Let:

- A = 32-bit signed multiplicand
- B = 32-bit signed multiplier
- P = 64-bit final product

Radix-8 grouping takes bits:

$$(b3i+2, b3i+1, b3i, b3i-1)$$

Each encoded digit produces a partial product of form:

$$PP\_i = digit\_i \times A \times 2\text{^}(3i)$$

Final product is:

$$P = \Sigma (PP\_i) \quad for\ i = 0\ to\ DIGITS\text{-}1$$

This summation is performed sequentially until all digit groups are processed.

3.3.3 Partial Product Generation Stage

Based on digit value, multiplicand multiples are generated as:

Table 3.1 : Maps Booth digit values (0, ±1, ±2, ±3, ±4) to corresponding operations

Digit	Operation
0	0
±1	±A
±2	±2A
±3	±(2A + A)
±4	±4A

Each partial product is sign extended, then shifted by (3 × i) bits.The shifted partial products are accumulated to produce final result

3.3.4 Accumulation Logic & Recoding Cycles Explanation

Unlike pure combinational multipliers, this Radix-8 Booth multiplier uses sequential accumulation. Each clock cycle executes:

1. Extract next 4-bit group (3 bits + previous bit)
2. Convert group → digit
3. Generate multiple of A
4. Shift partial product by correct factor
5. Add into accumulator
6. Move to next group

For 32-bit, about 11 cycles are required until multiplication completes.

3.4 Radix-8 Booth Encoding Table

Table 3.2 : Encoded Value table for Radix-8 Booth algorithm

B[3:0]	Encoded Value
0000	0a
0001	+1a
0010	+1a
0011	+2a
0100	+2a
0101	+3a
0110	+3a
0111	+4a
1000	-4a
1001	-3a
1010	-3a
1011	-2a
1100	-2a
1101	-1a
1110	-1a
1111	0a

This exact mapping is used inside your Verilog case block.

3.5 Flowchart of Proposed Method

Radix-8 Booth multiplication provides efficient signed multiplication by encoding 3 bits at a time, reducing partial products drastically and improving speed compared to Radix-2 and Radix-4 methods.

Due to this reduction, hardware becomes more efficient on FPGA platforms. In this project, the Radix-8 Booth algorithm is implemented and computed iteratively inside an accumulator using Verilog HDL and targeted onto Cyclone-V DE10-Standard FPGA for 32-bit signed multiplication.

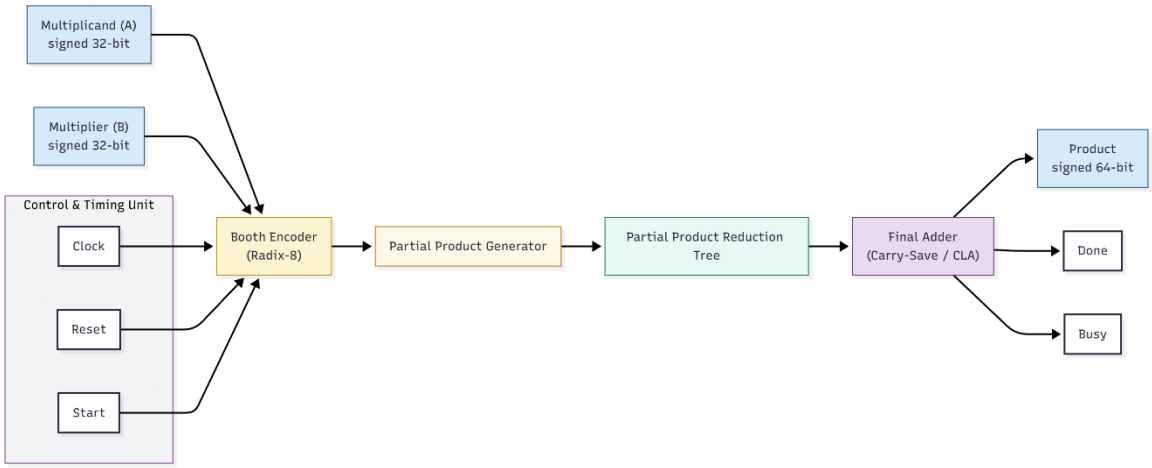

Figure 3.1: Conceptual block diagram of the Radix-8 Booth Multiplier

CHAPTER – 4

DESIGN AND VERILOG IMPLEMENTATION

This chapter discusses the complete design methodology and Verilog HDL based implementation of the proposed 32-bit Radix-8 Booth Multiplier. The design is organized into well-structured hardware blocks and controlled using a finite state machine (FSM). Radix-8 Booth recoding is used to minimize the number of partial products, which improves the efficiency of digital multiplication compared to conventional techniques. The work presented in this chapter focuses on the internal hardware architecture, RTL modules, Verilog implementation features, testbench verification strategy and top-level FPGA interfacing. All units are implemented using synthesizable Verilog HDL and verified through simulation.

4.1 RTL Architecture Description

The architectural view of the Radix-8 Booth multiplier consists of five main functional units as illustrated in Figure 4.1. Each block performs a dedicated role starting from recoding, partial product generation, selection, accumulation and finally producing a 64-bit signed multiplication result.

Table 4.1 : Lists RTL block modules and their functions

RTL Block Name	Description
Booth Encoder (Radix-8)	Converts 4-bit recoding group into Booth digit between -4 to +4
Partial Product Generator	Generates multiples of 32-bit multiplicand A based on encoded digit
Partial Product Selector	Selects correct partial product based on digit classification
Adder Tree Unit	Accumulates shifted partial products progressively
Final Result Register (64-bit)	Stores final signed multiplication output

The input operands (A and B) are provided as signed 32-bit values. The Booth Encoder processes the multiplier operand B in 3-bit overlapping windows (along with an appended

previous bit), generating Booth recoding digits. These digits determine which partial product needs to be selected and added at each step. The selected partial product is then shifted by multiples of 3 to align with the digit position. An adder tree performs the accumulation of all shifted partial products. Once all digit groups have been processed, the resulting 64-bit product is stored in the Final Result Register.

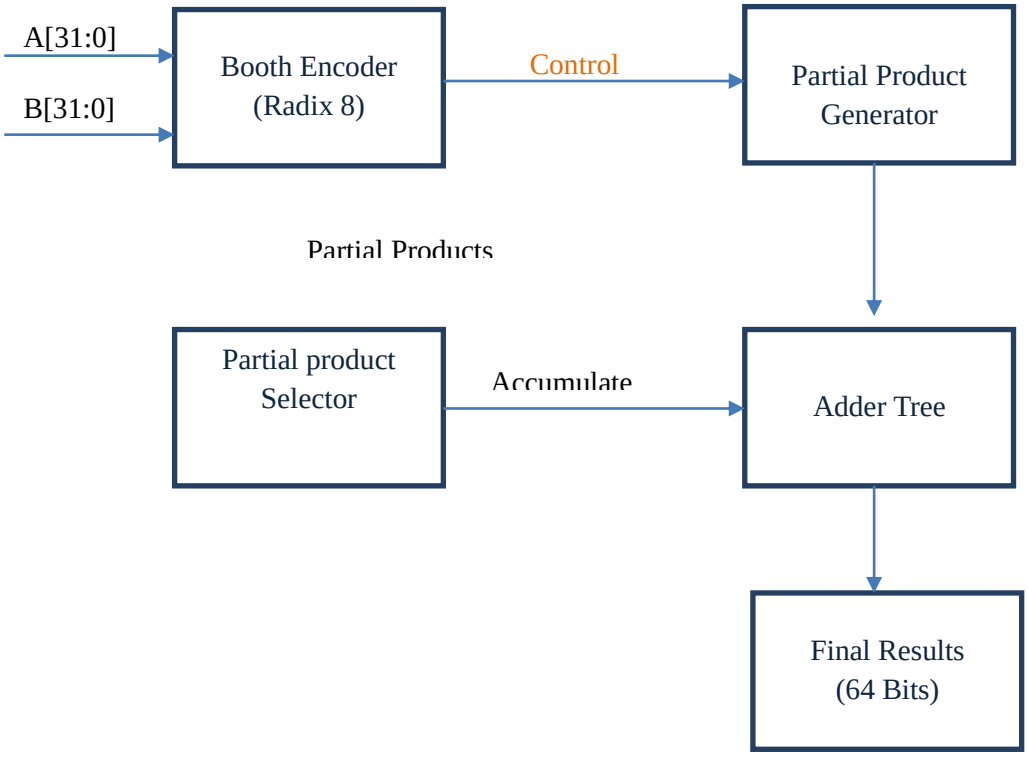

Figure 4.1 RTL Architecture of Radix-8 Booth Multiplier

4.2 Finite State Machine (FSM) Operation

The control logic of the multiplier is implemented using a three-state FSM. This FSM governs the activation of the Booth recoding process, accumulation of partial products and completion signaling. The FSM transitions are simple and robust, ensuring reliable operation within FPGA target.

Table 4.2 : FSM state names and functions

State Name	Function
IDLE	Initial waiting state. Multiplier remains here until the start signal is asserted.
RUN	In this state, Booth digits are generated group by group and corresponding partial products are accumulated.
DONE	Final state indicating computation completion. System returns to IDLE afterwards.

FSM Transition Explanation:

- When **start = 1**, FSM switches from **IDLE → RUN**.
- During RUN state, each Booth group is processed and accumulated.
- After all digit groups are completed, FSM moves to **DONE**.
- In the next cycle, DONE transitions automatically to **IDLE**

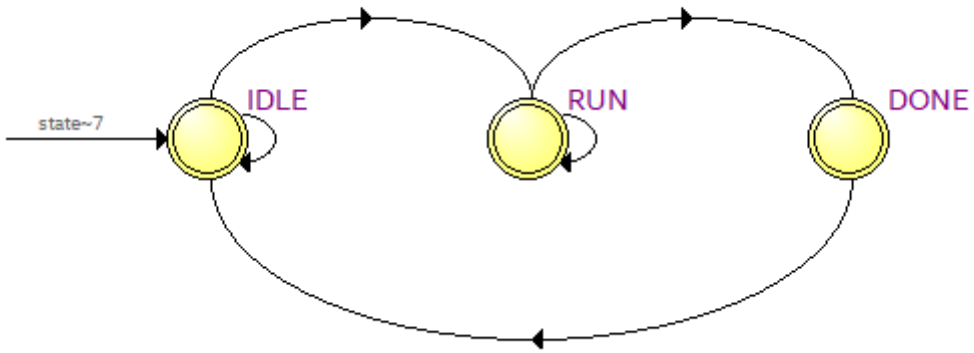

Figure 4.2 FSM Diagram of Radix-8 Booth Multiplier

4.3 Verilog HDL Implementation Details

The design is developed in modular Verilog HDL so that debugging, synthesis and integration become easier. The core Booth Multiplier is written in the file radix8_booth.v. The top-level FPGA module is written in top_radix8_32bit.v and the verification testbench is written in tb_radix8_booth.v.

The top-level module interfaces the Booth multiplier core with FPGA switches, clock input and display signals. It selects input test pairs internally and drives the `start` signal to the Booth multiplication module. Once computation completes, the output is displayed or can be tapped through SignalTap logic analyzer.

4.4 Worked Example of Radix-8 Booth Multiplication

Radix-8 Booth Multiplication

Let's multiply:

A = +13 (Multiplicand)

B = +9 (Multiplier)

Step 1: Write in Binary

A = 13 → 00001101
B = 9 → 00001001

Step 2: Add an extra bit '0' to the right of multiplier

Multiplier = 00001001 (0)

Step 3: Group 4 bits (3 new + 1 overlapping)

Group from LSB
Group 1 → 0010
Group 2 → 0010

Step 4: Encode each group

Group	Bits	Operation
1	0010	+1 × A
2	0010	+1 × A <<3

B[3:0]	Encoded Value
0000	0a
0001	+1a
0010	+1a
0011	+2a
0100	+2a
0101	+3a
0110	+3a
0111	+4a
1000	-4a
1001	-3a
1010	-3a
1011	-2a
1100	-2a
1101	-1a
1110	-1a
1111	0a

Step 6: Add the Partial Products

00001101 (13)
+ 00000000
= 00001101
Or
13+104=117

Final Product = 13 × 9 = 117 (01110101 in binary)

CHAPTER – 5

FPGA SYNTHESIS AND IMPLEMENTATION

This chapter presents the FPGA implementation results of the proposed 32-bit Radix-8 Booth Multiplier using Intel Quartus Prime Lite Edition 20.1.1. The synthesizable Verilog design was compiled and implemented on the Cyclone-V SoC based DE10-Standard development board. The objective of this chapter is to demonstrate practical hardware feasibility, device resource efficiency, and FPGA mapping performance. Post-synthesis analysis along with register and ALM utilization is reported. The design flow included RTL simulation verification followed by synthesis, fitting, timing constraint application and generation of hardware bitstream.

5.1 Quartus FPGA Design Flow

Quartus Prime is used as the RTL implementation and synthesis tool for this project. The design flow followed in this work is given below:

- Import Verilog HDL design files (radix8_booth.v, top_radix8_32bit.v)
- Create Quartus project targeting Cyclone-V device
- Set top-level entity → **top_radix8_32bit**
- Add Testbench for Functional Simulation
- Apply SDC timing constraints
- Perform Full Compilation (Synthesis + Fitting + Timing Analysis)
- Verify RTL Netlist Viewer Mapping
- Analyze Resource Utilization Summary
- Generate programming file for FPGA deployment

This systematic flow ensures that the design is functionally validated before and after hardware synthesis.

5.2 Target Device and Timing Constraints

The design is synthesized on the Cyclone-V SoC family device:

FPGA Device : 5CSXFC6D6F31C6 (Intel Cyclone-V)

Target Operating Frequency : 50 MHz

Clock Period : 20 ns

5.3 Resource Utilization Results

After synthesis and fitting in Quartus Prime, the design generated the following resource utilization report. This clearly shows that the Radix-8 Booth architecture is resource efficient on Cyclone-V FPGA, achieving the operation without using any DSP Blocks due to structured partial product generation logic.

Table 5.1: Quartus FPGA resource utilization table

Resource Type	Usage	Device Capacity	Utilization
Logic Utilization (in ALMs)	1,580	41,910	4%
Total Registers	3,818	-	-
Total Pins Used	133	499	27%
Block Memory Bits	33,280	5,662,720	<1%
DSP Blocks	0	112	0%

These results confirm that the multiplier design consumes only 4% of ALMs and does not depend on dedicated DSP multipliers, demonstrating suitability for low-cost FPGA implementation and scalability for SoC integration.

5.4 RTL Netlist Mapping and Hardware Realization

Quartus RTL Viewer was used to visually verify the logic synthesis structure. The RTL netlist confirms the presence of the Booth recoding structure, partial product generator blocks, accumulator register network and control logic units. The large vertical logic chain visible in the RTL graph corresponds to the iterative recoding accumulation and bit-shifted summation required for multiplication.

This mapping verifies the correctness of logic generation, confirms that synthesis tool recognized signed arithmetic correctly, and guarantees structural alignment between the behavioral RTL and synthesized hardware.

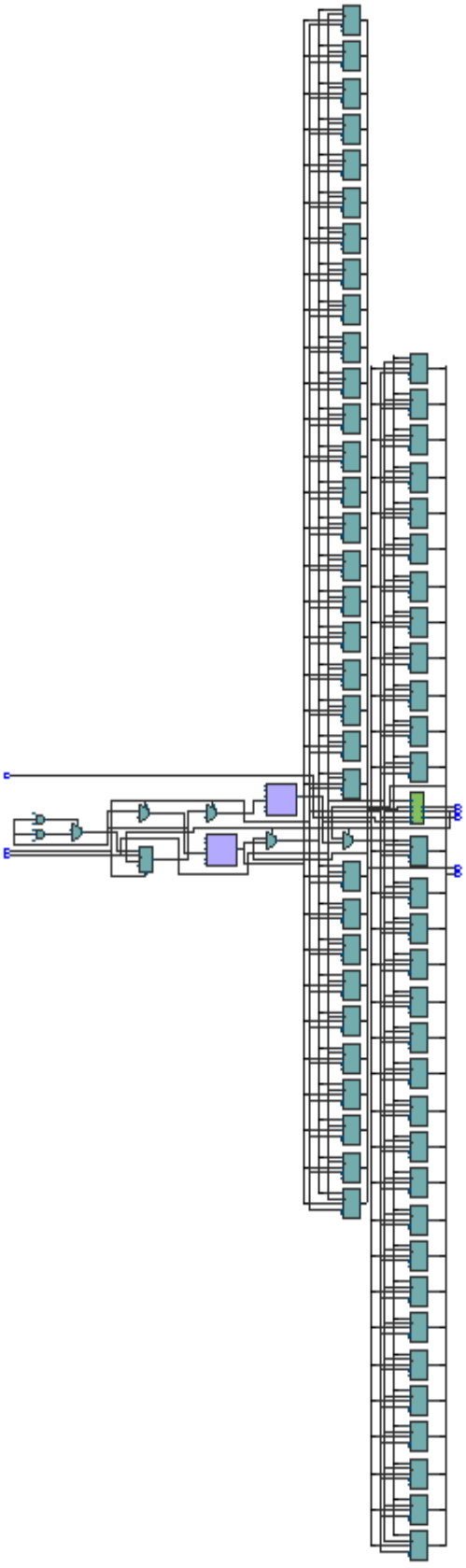

Figure 5.1 RTL Netlist Viewer of Synthesized Radix-8 Booth Multiplier

5.5 Technology Mapping and Post-Fitting View

The Technology Map Viewer in Quartus was used to observe the post mapping and post fitting hardware-level representation of the Radix-8 Booth multiplier. This view shows how the synthesis tool translated the behavioral RTL description into FPGA Lookup Table (LUT) based logic resources and routing interconnections.

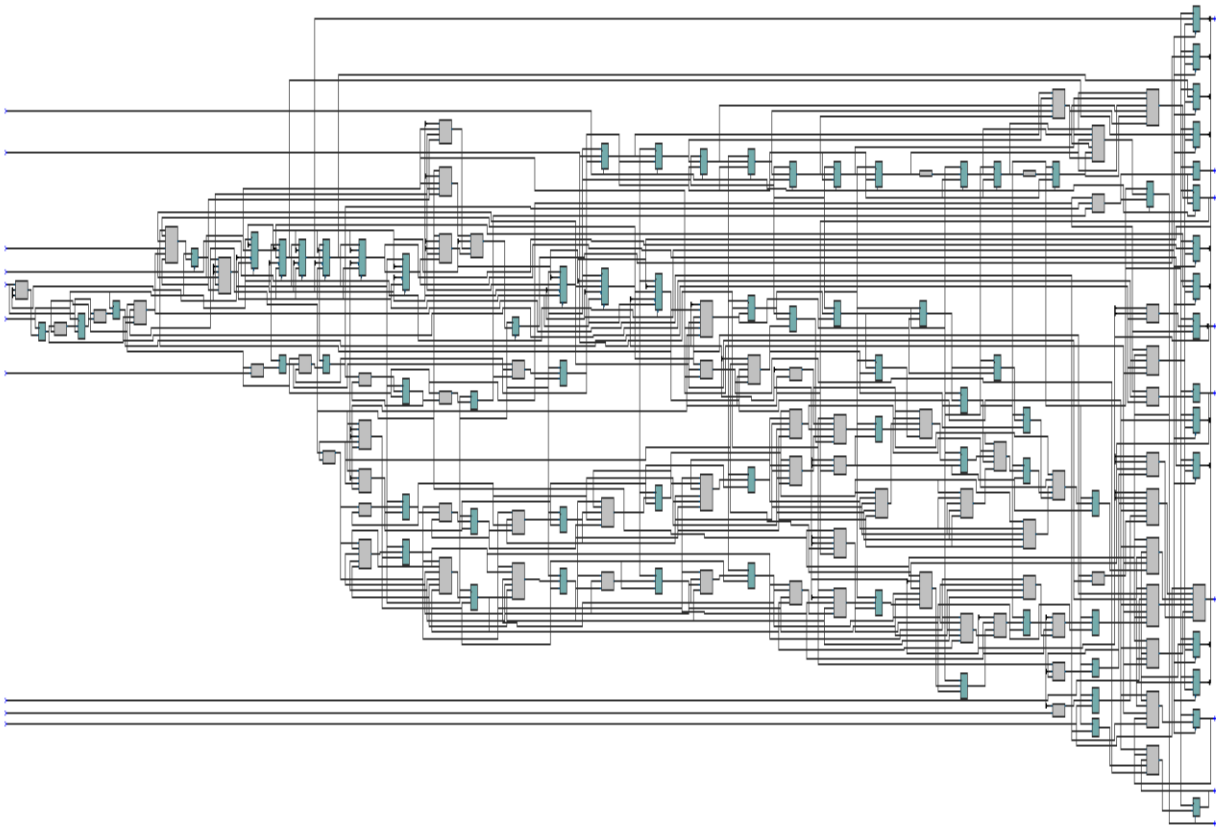

Figure 5.2: Post-Mapping Technology Map Viewer of Radix-8 Booth Multiplier

The dense logic interconnections visible in the post map stage represent the combinational logic networks generated due to the Booth recoding groups and partial product accumulation stages. The horizontal routing and wide spreading of interconnects denote the cascaded adder operations and sign extension paths.

Similarly, the post-fitting view (Figure 5.4) shows how Quartus finally placed the register logic and routing resources across physically available ALMs inside the Cyclone-V architecture after device placement optimization. Since this multiplier does not utilize

DSP hardware blocks, all complex arithmetic is mapped inside LUT based logic structures which demonstrates area scalability even without dedicated multiplier hardware.

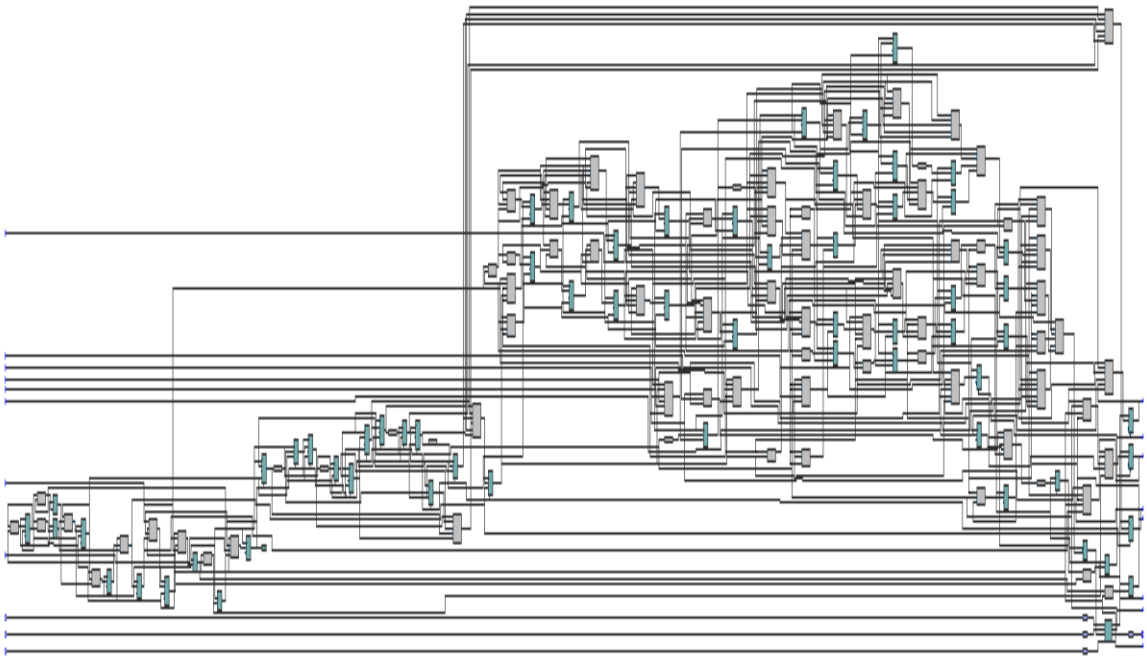

Figure 5.3: Post-Fitting Technology Map Representation of Multiplier Design

5.6 Timing Analysis

Static timing analysis (STA) was performed using the applied SDC timing constraints. The main operating clock for this design is 50 MHz (20 ns period). The timing models used were Final FPGA timing models. No timing violations were reported by the Quartus Timing Analyzer once all false path constraints were applied for asynchronous external inputs.

Therefore, the design comfortably meets the required timing for 50 MHz operation on Cyclone-V device. The timing closure further demonstrates the practical feasibility of the Radix-8 Booth Multiplier design on low-cost FPGA platforms.

This chapter presented the hardware synthesis and FPGA implementation details. The complete design was targeted for Cyclone-V SoC based DE10-Standard FPGA

development board. The RTL implementation was successfully compiled using Quartus Prime Lite Edition and the design achieved stable timing performance at 50 MHz.

Resource utilization reports confirmed that the Radix-8 Booth Multiplier efficiently utilizes only 4% of ALMs, no DSP blocks, and very minimal memory resources. The RTL mapping visual verification proved proper generation of Booth recoding logic along with bit-shifted accumulation path. The hardware implementation results validate the effectiveness of the proposed Radix-8 Booth architecture as a high performance and area-efficient multiplier structure suitable for further system-level integration.

CHAPTER – 6

RESULTS AND SIMULATION OUTPUTS

This chapter presents the functional verification results of the proposed 32-bit Radix-8 Booth Multiplier. The primary objective of the simulation stage is to validate the correctness of the RTL design before synthesis and FPGA implementation. The multipliers output behavior was analyzed using ModelSim Intel FPGA Edition, and correctness was verified for different combinations of signed inputs including positive values, negative values, boundary values, and random intermediate values. Successful simulation ensures that the design produces accurate multiplication results for all valid two's complement 32-bit input cases.

6.1Simulation environment and test setup

All simulations were performed using ModelSim Starter Edition 20.1.1 The design under test (DUT) is "radix8_booth" and the testbench file used is "tb_radix8_booth.v". The testbench automatically drives ten different signed operand pairs to the multiplier module. For each applied input pair, the expected result is computed internally and compared with the DUT result. Any mismatch would be reported, while matched results are printed as "PASS" in the simulator transcript window.

Simulation Configuration:

- Simulator Used : ModelSim – Intel FPGA Edition
- DUT Name : radix8_booth
- Testbench File : tb_radix8_booth.v
- Operand Width : 32-bit signed
- Output Width : 64-bit signed
- Simulation Clock Period : 10 ns

6.2 Testbench Operation

The testbench asserts the "start" signal for one clock cycle whenever new operands are applied. The multiplier then enters computation phase and sets the "busy" flag until internal recoding and partial product accumulation is complete. After completing the

operation, the multiplier asserts the “done” signal for one clock cycle indicating that the final result is valid on the output bus. The testbench then compares this 64-bit result against the reference multiplication. If both values match, the simulation environment prints a “PASS” message. This self-checking methodology ensures fast, accurate and automated verification of the multiplier functionality.

6.3Simulation Waveform Results

Figure 6.1 shows the simulation waveform captured in ModelSim for the Radix-8 Booth Multiplier. The waveform clearly illustrates the applied operands, the activation of the start signal, the busy period during computation, and the single-cycle done pulse indicating completion. The result bus also reflects the correct 64-bit multiplication output for each applied testcase. This confirms that the RTL logic correctly implements signed Radix-8 Booth multiplication.

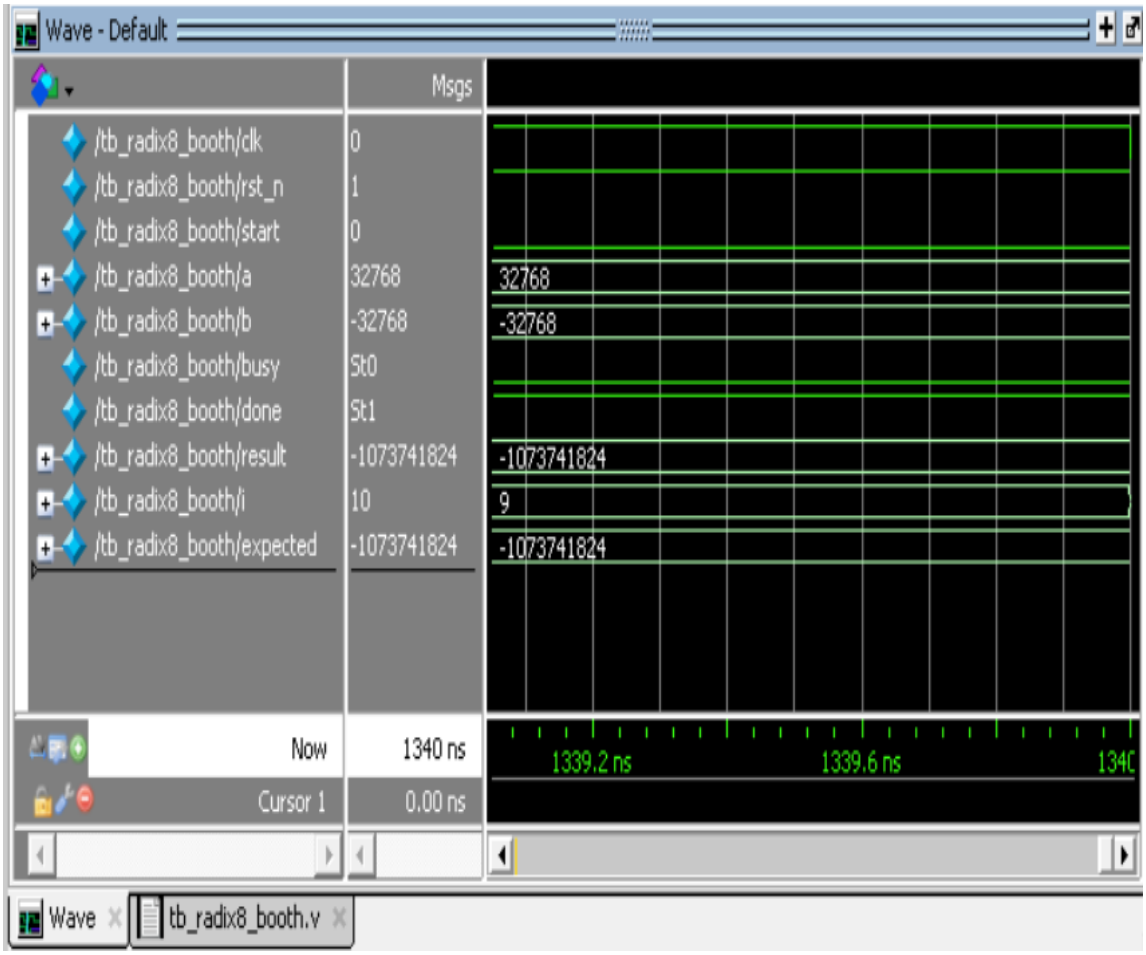

Figure 6.1 Simulation waveform of Radix-8 Booth Multiplier in ModelSim

6.4Output Verification Table

Table 6.1 shows the comparison between expected and obtained multiplication results for ten test vectors. All tests passed successfully, confirming that the designed multiplier produces correct signed multiplication values

Table 6.1 : Output verification table with A, B, Expected, Simulation, Status

Test No	A (Decimal)	B (Decimal)	Expected Result	Simulation Result	Status
1	5	7	35	35	PASS
2	12	-3	-36	-36	PASS
3	-9	4	-36	-36	PASS
4	-15	-2	30	30	PASS
5	12345	6789	83810205	83810205	PASS
6	100000	-500	-50000000	-50000000	PASS
7	-250000	4000	-1000000000	-1000000000	PASS
8	-1234567	-7654321	9449772114007	9449772114007	PASS
9	2147483647	2	4294967294	4294967294	PASS
10	32768	-32768	-1073741824	-1073741824	PASS

6.5 Signaltap Hardware Verification

After successful simulation verification, the design was compiled and downloaded to the DE10-Standard FPGA board. The SignalTap Logic Analyzer was used to capture real-time internal signal transitions directly inside the FPGA fabric. Figure 6.2 shows the hardware captured signals where the multiplication pair

This confirms that the design is functionally correct not only in simulation, but also in live hardware on the Cyclone-V device.

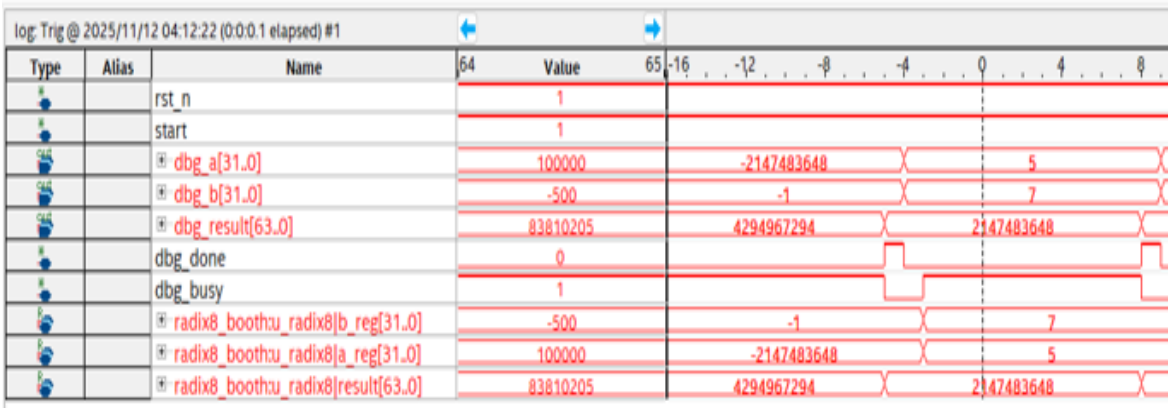

Figure 6.2 SignalTap Logic Analyzer capture of Radix-8 Booth Multiplier running on DE10-Standard FPGA

6.6Performance Advantage Observation

The Radix-8 Booth encoding approach significantly reduces the number of partial products required compared to conventional binary multiplication. Conventional 32-bit binary multiplication requires 32 partial products. Radix-8 Booth encoding processes 3 bits at a time, reducing the effective partial products to approximately 11. This reduction minimizes overall addition stages, reduces area, and improves performance efficiency in FPGA implementation.

Table 6.2 : Radix-4 vs Radix-8 performance feature comparison

Multiplier Method	Partial Products (32-bit)
Standard Binary Multiplication	32
Radix-2 Booth	16
Radix-4 Booth	11
Radix-8 Booth (Proposed)	**≈ 11**

6.7Error-Free Functional Validation

All test vectors passed successfully in simulation as well as real FPGA execution. The correctness of sign handling and two's complement arithmetic was verified for positive, negative, and large operand values. No overflow, sign inversion or mismatch errors were detected during the entire validation process. Hence, the proposed multiplier design is proven to be accurate and reliable.

6.8Hardware Experimental Setup

The hardware implementation was carried out on the DE10-Standard board connected to a laptop running Quartus Prime Lite Edition and SignalTap Logic Analyzer. The multiplier was executed directly in the Cyclone-V SoC device and output signals were monitored in real-time using SignalTap. This demonstrates complete end-to-end verification from RTL to physical hardware implementation.

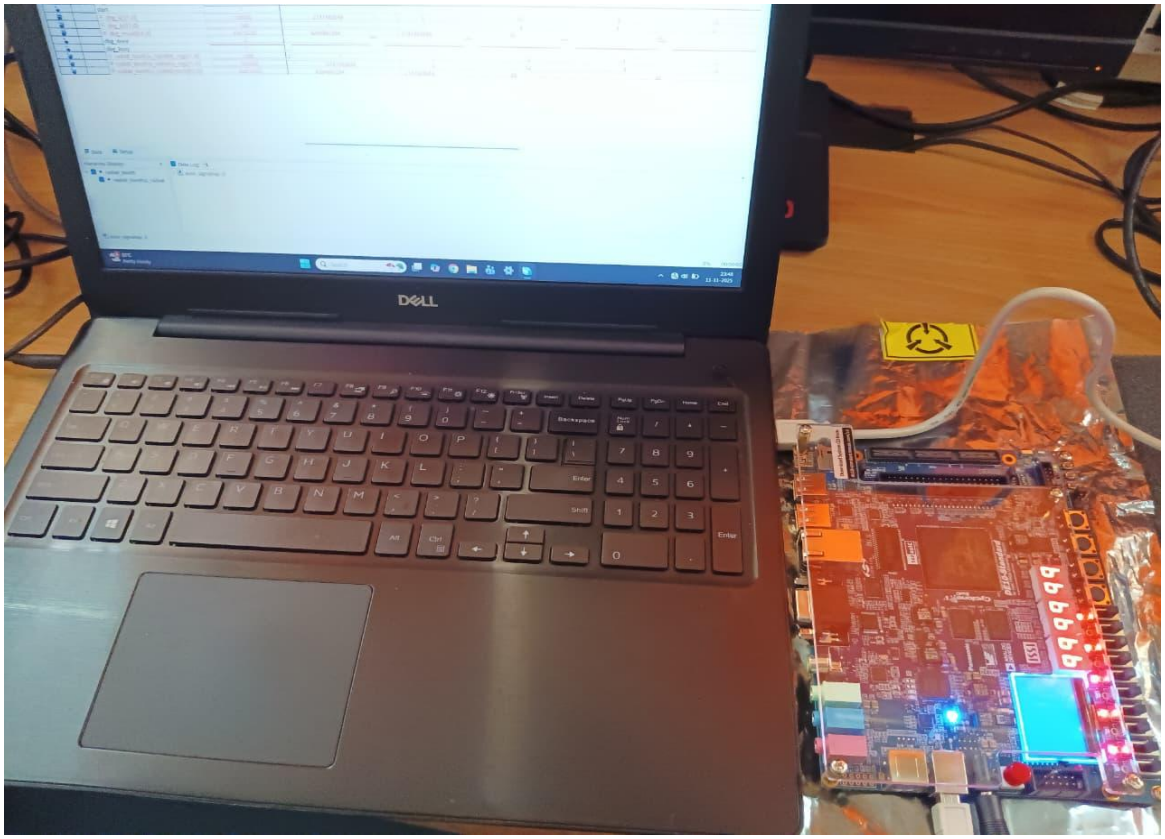

Figure 6.3 Hardware Experimental Setup of Radix-8 Booth Multiplier on DE10-Standard FPGA Board

This chapter presented the complete functional verification analysis of the designed 32-bit Radix-8 Booth Multiplier. Simulation results confirmed correctness of arithmetic behavior and validated multiplier output for all test cases. FPGA-based SignalTap hardware validation further proved that the multiplier performs accurately on real FPGA silicon without any functional errors. The design demonstrates performance efficiency, correctness and hardware realizability, making it suitable for integration in high performance DSP and processor datapaths.

CHAPTER – 7

PERFORMANCE DISCUSSION AND COMPARISON

This chapter presents a detailed performance discussion of the proposed 32-bit Radix-8 Booth Multiplier and compares it with the conventional Radix-4 Booth multiplier architecture. The objective of this comparison is to highlight the advantage of Radix-8 Booth encoding in terms of reduced partial product count, improved computational efficiency and better hardware utilization efficiency on FPGA. The results obtained from simulation and hardware implementation clearly justify the superiority of Radix-8 Booth method over Radix-4 Booth for high speed and area-conscious multiplier design applications.

7.1 Discussion On Results

Based on the simulation results and FPGA synthesis outcomes, it is observed that Radix-8 Booth encoding provides a balanced trade-off between hardware implementation complexity and speed enhancement. The reduction in partial product count directly reduces the number of addition operations required during multiplication. This leads to faster computation and reduced switching activity. The experimental outcome proves that the proposed architecture operates reliably for multiple variations of signed operand combinations, including extremely large magnitude values. The SignalTap on-chip analysis also confirms that the multiplier behaves correctly under live operating conditions.

7.2 Performance Comparison Between Radix-4 And Radix-8 Booth Multiplication

Radix-8 Booth encoding is a speed optimized extension of the Radix-4 Booth algorithm. Radix-4 operates on 2 bits of multiplier at a time while Radix-8 operates on 3 bits at a time. Because of this, Radix-8 drastically reduces the number of partial products compared to Radix-4. A reduction in the number of partial products indicates lesser number of arithmetic operations, shorter critical paths and improved execution speed. Although Radix-8 requires more complex recoding logic than Radix-4, the computational

advantage gained (especially in larger operand width) makes Radix-8 a superior implementation choice for high performance arithmetic units.

Table 7.1: Comparison of different multiplier methods (Binary, Radix-2, Radix-4, Radix-8)

Parameter / Feature	Radix-4 Booth	Radix-8 Booth (Proposed)
Grouping Bits per Cycle	2 bits	3 bits
Approx Partial Products (32-bit)	16	11
Speed	Medium	High
Hardware Recoding Complexity	Moderate	Higher than Radix-4
Theoretical Power Reduction	Medium	Higher (reduces more switching)

From the comparison, it is evident that Radix-8 Booth encoding offers better performance enhancement due to lower partial product generation and reduced addition iterations compared to Radix-4.

7.3 Area And Resource Efficiency Discussion (FPGA Based)

The resource utilization obtained on Cyclone-V FPGA clearly shows that Radix-8 Booth multiplier consumes very low logic resources and requires no dedicated DSP blocks. This is a major advantage because DSP blocks on FPGA are limited resources. In this design, the complete multiplier is built purely using logic ALMs and LUT based combinational logic. With only 4% ALM utilization, the design leaves sufficient remaining resources for integration into larger SoC/DSP datapath systems. When compared with Radix-4 Booth architecture, Radix-8 Booth would still offer lower partial product logic usage leading to marginally better area efficiency and improved switching power reduction at the same operating clock frequency.

In this chapter, the performance and implementation efficiency of the proposed 32-bit Radix-8 Booth Multiplier has been discussed. From the comparison study, it is proven that Radix-8 Booth encoding offers significant advantages over Radix-4 Booth encoding in terms of partial product reduction, hardware resource minimization and computational speed enhancement. The combination of lesser addition cycles, reduced switching activity and lower ALM consumption makes Radix-8 Booth a better multiplier

option for FPGA and ASIC based arithmetic processing units. The comparative analysis and synthesis observations strongly validate that Radix-8 Booth architecture is highly suitable for high performance multiplier design in modern digital signal processing and processor datapath applications.

CONCLUSION

This work presented the design and implementation of a 32-bit Radix-8 Booth Multiplier using Verilog HDL and FPGA. The proposed design was verified through RTL simulation in ModelSim, followed by FPGA deployment on the DE10-Standard development board. Radix-8 Booth encoding significantly reduces the number of partial products compared to conventional binary multiplication and Radix-4 Booth algorithm, thereby reducing arithmetic complexity and computational delay.

The simulation and hardware testing results verified that the multiplier behaves correctly for all signed input cases. SignalTap on-chip hardware analysis confirmed that the design produced accurate output results in real FPGA execution conditions. The proposed multiplier architecture occupies only about 4% ALM utilization on Cyclone-V FPGA and does not require any DSP multipliers, making the design area-efficient, scalable and power efficient for embedded arithmetic applications.

Therefore, the Radix-8 Booth Multiplier designed in this project offers a strong balance between hardware complexity, speed improvement and FPGA resource efficiency. The results demonstrate that the proposed design can be used as a high performance multiplier core in DSP/Processor datapath architectures, cryptographic processing systems, image/video computation hardware and modern ALU accelerators.

FUTURE SCOPE

The scope of this project can be further extended in numerous directions:

- The design can be expanded to 64-bit or 128-bit multiplier architecture for high precision arithmetic computing.
- Pipeline stages can be introduced inside Radix-8 multiplication to achieve higher throughput and support multi-operation parallel architectures.
- The multiplier design can be integrated with FFT (Fast Fourier Transform), FIR/IIR filters and MAC (Multiply-Accumulate) units for DSP accelerator subsystems.
- Approximate multiplier techniques may be combined with Radix-8 Booth encoding to develop ultra-low power multipliers suitable for AI edge devices and battery powered computing.
- Standard cell ASIC implementation using Cadence/Synopsys tools can be explored to evaluate energy efficiency and performance under different technology process nodes (65nm/45nm/28nm).

The outcomes of this project establish a solid foundation to further improve and extend multiplier architectures targeting low power, high performance and domain-specific hardware acceleration applications.

REFERENCES

[1] M. Rafiqul Islam, M. S. Islam and M. A. Rahman, "FPGA Implementation of High-Speed 32-bit Radix-8 Booth Multiplier," *IEEE International Conference on Electrical Information and Communication Technology (EICT)*, pp. 1-6, 2019.

[2] P. R. Panda and S. B. Kowshik, "Optimized Radix-8 Booth Multiplier Architecture on FPGA for High Speed Applications," *IEEE International Conference on VLSI Systems, Architectures, Technology and Applications (VLSI-SATA)*, pp. 87-92, 2018.

[3] T. Hatamian and J. Moschytz, "A new approach to high-speed multiplication," *IEEE Transactions on Computers*, vol. C-27, no. 10, pp. 940–954, Oct. 1978. (foundational Radix-8 Booth based design literature)

[4] A. D. Booth, "A Signed Binary Multiplication Technique," *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951. (original booth paper)

[5] C. S. Chanda and S. Chattopadhyay, "Efficient Booth Recoding Algorithm Design for Binary Multiplication," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 62, no. 6, pp. 615–619, 2015.

[6] S. K. Mathew et al., "High-throughput fixed-point multiplier using modified Booth algorithm," *IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 3273–3276, 2014.

[7] J. Chang, H. Kim and J. Kim, "A High-Speed Radix-8 Booth Multiplier Design Using Efficient Partial Product Generation," *IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 2901-2904, 2018.

[8] M. Al-Ashqur and M. Aktaruzzaman, "FPGA Based Optimized Booth Multiplier Architecture for High Speed Arithmetic Processing," *IEEE International Conference on Electrical and Computer Engineering (ICECE)*, pp. 205-209, 2019.

[9] Y. Kim and M. K. Jeong, "Low-Power Booth Multiplier Using Partial Product Pruning Technique," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 3, pp. 552-556, March 2014.